

Universidad de San Carlos de Guatemala
Escuela de Ciencias y Sistemas
Facultad de Ingeniería
Lenguajes Formales y de Programación
Primer Semestre 2026

MANUAL TÉCNICO

Gabriel Eduardo Urbina Sunún
Carnet: 202300384
Cui: 2794761080101

1. Arquitectura del Sistema

El sistema está desarrollado en C++ estándar (C++17) con arquitectura monolítica de un solo archivo fuente. Se organiza en capas lógicas bien diferenciadas que separan la gestión de datos, la lógica de negocio y la presentación.

1.1 Organización del Código Fuente

Sección	Líneas aprox.	Responsabilidad
Estructuras de Datos	1-30	Define Estudiante, Curso y Nota
Variables Globales	31-50	Vectores principales de datos en memoria
Funciones Utilitarias	51-110	split(), calcularMediana(), calcularDesviacionEstandar(), buscarEstudiante(), buscarCurso()
Carga de Archivos	111-200	cargarEstudiantes(), cargarCursos(), cargarNotas()
Generación de Reportes	201-500	5 funciones generadoras de HTML
Menú y Main	501-560	mostrarMenu(), main()

2. Estructuras de Datos

El sistema utiliza tres estructuras primarias almacenadas en vectores globales. Todo el procesamiento se realiza en memoria RAM durante la ejecución.

2.1 struct Estudiante

```
struct Estudiante {  
    int    carnet;        // Identificador único del estudiante  
    string nombre;        // Primer nombre  
    string apellido;      // Apellido  
    string carrera;        // Nombre de la carrera  
    int    semestre;      // Semestre actual  
};
```

2.2 struct Curso

```
struct Curso {  
    int    codigo;        // Código único del curso  
    string nombre;        // Nombre descriptivo del curso  
    int    creditos;       // Créditos académicos del curso  
    int    semestre;      // Semestre al que pertenece  
    string carrera;        // Carrera a la que pertenece  
};
```

2.3 struct Nota

```
struct Nota {  
    int    carnet;        // FK -> Estudiante  
    int    codigo_curso;  // FK -> Curso  
    double nota;          // Calificación (0.0 - 100.0)  
    string ciclo;         // Ciclo académico ('1' o '2')  
    int    anio;          // Año académico  
};
```

2.4 Almacenamiento en Memoria

Los datos se almacenan en tres vectores globales:

```
vector<Estudiante> estudiantes;  
vector<Curso>      cursos;  
vector<Nota>       notas;
```

No existe persistencia entre ejecuciones. Los datos se pierden al cerrar el programa.

3. Descripción de Funciones

3.1 Funciones Utilitarias

split(string, char) -> vector<string>

Tokeniza una cadena de texto usando un delimitador. Elimina espacios en blanco al inicio y final de cada token usando find_first_not_of y find_last_not_of.

- Entrada: cadena completa + carácter delimitador
- Salida: vector de subcadenas limpias
- Complejidad: $O(n)$ donde n = longitud de la cadena

calcularMediana(vector<double>&) -> double

Ordena el vector in-place y retorna el valor central. Para conjuntos pares retorna el promedio de los dos valores centrales.

- NOTA: Modifica el vector original (ordena ascendente)
- Retorna 0.0 si el vector está vacío

calcularDesviacionEstandar(vector<double>&, double) -> double

Calcula la desviación estándar poblacional (divide entre N , no $N-1$).

- Utiliza la fórmula: $\sqrt{\text{suma}((x - \text{media})^2) / N}$
- Retorna 0.0 si hay menos de 2 elementos

buscarEstudiante(int) -> Estudiante*

Búsqueda lineal $O(n)$ sobre el vector global. Retorna puntero al estudiante o nullptr si no existe.

buscarCurso(int) -> Curso*

Búsqueda lineal $O(n)$ sobre el vector global. Retorna puntero al curso o nullptr si no existe.

3.2 Funciones de Carga

Las tres funciones de carga (`cargarEstudiantes`, `cargarCursos`, `cargarNotas`) siguen el mismo patrón:

1. Solicita nombre del archivo por `stdin`
2. Abre el archivo con `ifstream`
3. Limpia el vector global correspondiente (`.clear()`)
4. Lee línea por línea con `getline()`
5. Valida que haya al menos 5 campos por línea
6. Convierte strings a tipos nativos (`stoi`, `stod`)
7. Agrega al vector con `push_back()`

ADVERTENCIA: No hay manejo de excepciones para `stoi/stod`. Un archivo con datos no numéricos en campos enteros o decimales lanzará una excepción `std::invalid_argument` no capturada.

3.3 Funciones de Reporte

Función	Datos Requeridos	Archivo Generado
<code>generarReporteEstadisticasCurso()</code>	cursos + notas	<code>reporte_estadisticas_curso.html</code>
<code>generarReporteRendimientoEstudiante()</code>	estudiantes + notas	<code>reporte_rendimiento_estudiante.html</code>
<code>generarReporteTop10()</code>	estudiantes + notas	<code>reporte_top10.html</code>
<code>generarReporteReprobacion()</code>	cursos + notas	<code>reporte_reprobacion.html</code>
<code>generarReporteAnalisisCarrera()</code>	estudiantes + cursos + notas	<code>reporte_analisis_carrera.html</code>

4. Algoritmos Clave

4.1 Cálculo de Estadísticas por Curso

Para cada curso, el sistema itera linealmente sobre todo el vector de notas buscando coincidencias por `codigo_curso`. Complejidad: $O(\text{cursos} \times \text{notas})$.

```
Para cada curso:
  Para cada nota:
    Si nota.codigo_curso == curso.codigo:
      Agregar a coleccion local
  Calcular: promedio, max, min, desv.est., mediana
```

4.2 Ordenamiento para Top 10

Se construye un vector de pares (promedio, Estudiante) y se ordena con `std::sort` usando una lambda comparadora descendente. Se toman los primeros 10 elementos con `min(10, size)`.

4.3 Clasificación de Reprobación

Umbral de aprobación: `nota >= 61`.

- Alta reprobación: `porcentaje_reprobados >= 50%`
- Media reprobación: `30% <= porcentaje_reprobados < 50%`
- Baja reprobación: `porcentaje_reprobados < 30%`

5. Dependencias y Bibliotecas

Librería	Uso en el sistema
<code><iostream></code>	Entrada/salida estándar (cin, cout)
<code><fstream></code>	Lectura de archivos (ifstream) y escritura HTML (ofstream)
<code><sstream></code>	Tokenización de strings con stringstream
<code><vector></code>	Contenedor dinámico para datos en memoria
<code><string></code>	Manejo de cadenas de texto
<code><algorithm></code>	sort() para ordenamiento y min()
<code><cmath></code>	sqrt() y pow() para estadísticas
<code><iomanip></code>	fixed y setprecision() para formato de decimales
<code><map></code>	Agrupación por carrera en Reporte 5
<code><set></code>	Incluida pero no utilizada actualmente

6. Limitaciones y Mejoras Sugeridas

6.1 Limitaciones Actuales

- Sin persistencia de datos: los datos se pierden al cerrar el programa
- Búsquedas lineales $O(n)$: el rendimiento degrada con grandes volúmenes de datos
- Sin validación de datos duplicados al cargar archivos
- No hay manejo robusto de excepciones para conversiones stoi/stod
- La biblioteca `<set>` está incluida pero no se utiliza
- La desviación estándar usa N (poblacional) en lugar de $N-1$ (muestral)

6.2 Mejoras Recomendadas

- Reemplazar búsquedas lineales con `std::unordered_map` para $O(1)$ promedio
- Agregar try-catch alrededor de `stoi()` y `stod()` para manejo de errores
- Implementar validación de duplicados por carnet/código al cargar datos
- Agregar soporte para exportación a CSV además de HTML
- Considerar uso de base de datos SQLite para persistencia entre sesiones
- Separar el código en múltiples archivos (.h y .cpp) para mejor mantenibilidad